

Configuration Guide

ESL Data Pipeline

19.03.2026

João Pires - Devoteam

Sonae MC

Contents

- 1. ESL Data Pipeline – Configuration Guide 1
 - 1.1. 1. Changing the Schedule 1
 - 1.1.1. Examples 1
 - 1.1.2. How to change it 1
 - 1.2. 2. Filtering Stores 2
 - 1.2.1. Where to configure 2
 - 1.2.2. filter_stores 2
 - 1.2.3. filter_retail_chains 3
 - 1.2.4. Priority rules 3
 - 1.3. 3. Current pre-production configuration (reference) 3
 - 1.4. 4. Applying changes 4
 - 1.4.1. Git Workflow 4
 - 1.4.2. Step-by-step 4

1. ESL Data Pipeline – Configuration Guide

This guide explains how to change the execution schedule and store filters for the ESL Data Pipeline CronJob running in the pre-production cluster.

All changes are made in a single file inside the repository [lacIO4I-instore-orchestrator](#):

clusters/cluster-preproduction/values-pp.yaml

Look for the section starting with - name: orchestrator-esl-datapipeline (near the bottom of the file, under cronjobs:).

1.1. 1. Changing the Schedule

The schedule field controls when the pipeline runs. It uses standard cron syntax:



All times are in UTC.

1.1.1. Examples

Schedule	Meaning
"30 16 * * *"	Every day at 16:30 UTC (<i>current value</i>)
"0 6 * * *"	Every day at 06:00 UTC
"0 */6 * * *"	Every 6 hours
"0 8 * * 1-5"	Weekdays at 08:00 UTC
"0 2 * * 0"	Every Sunday at 02:00 UTC

1.1.2. How to change it

Find and update this field:

```
- name: orchestrator-esl-datapipeline
  ...
  schedule: "30 16 * * *" # <-- change this value
  timeZone: "UTC"
```

Example – change to run every day at 06:00 UTC:

```
schedule: "0 6 * * *"
```

Note: concurrencyPolicy: Forbid is already set, which means a new run will be skipped if the previous one is still in progress. There is no risk of overlapping executions.

1.2. 2. Filtering Stores

The filter_stores and filter_retail_chains settings allow you to restrict which stores the pipeline processes. This is useful for testing or for running the pipeline against a specific subset of stores.

1.2.1. Where to configure

These settings are inside the settings.values.connector block:

```
settings:
  fileVolumePath: /app/config.yaml
  fileName: config.yaml
  type: yaml
  values:
    connector:
      filter_stores:
        - "bomdia_pt.009648" # <-- list of store IDs to process
```

1.2.2. filter_stores

A list of specific store IDs to process. The store ID format is <retail_chain_id>.<store_number>.

Examples:

Process a single store:

```
filter_stores:
  - "continente_pt.001234"
```

Process multiple stores:

```
filter_stores:
  - "continente_pt.001234"
  - "continente_pt.005678"
  - "bomdia_pt.009648"
```

Process all stores (remove the filter entirely or leave as an empty list):

```
filter_stores: []
```

1.2.3. filter_retail_chains

A list of retail chain IDs to process — all stores belonging to those chains will be included.

Important: `filter_stores` takes priority. If `filter_stores` contains any entries, `filter_retail_chains` is ignored. Only set one or the other.

Example – process all stores from a specific retail chain:

```
filter_stores: []           # must be empty for filter_retail_chains
                             to take effect
filter_retail_chains:
  - "continente_pt"
```

Example – process stores from multiple chains:

```
filter_stores: []
filter_retail_chains:
  - "continente_pt"
  - "bomdia_pt"
```

1.2.4. Priority rules

filter_stores	filter_retail_chains	Result
Has entries	Any	Only the listed stores are processed
Empty	Has entries	All stores of the listed chains are processed
Empty	Empty	All stores are processed

1.3. 3. Current pre-production configuration (reference)

```
schedule: "30 16 * * *"    # Daily at 16:30 UTC
timeZone: "UTC"

settings:
  values:
```

```
connector:
  filter_stores:
    - "bomdia_pt.009648" # single store filter active
```

1.4. 4. Applying changes

Changes are applied through a pull request workflow on the [lac1041-instore-orchestrator](#) repository. ArgoCD syncs automatically once a change is merged into the app's target revision branch.

1.4.1. Git Workflow

- Branches must be created from the ArgoCD app target revision branch and the branch name must match: `feature/<description>` or `revert/<description>`
- Commit messages must follow [Conventional Commits](#):
 - `feat:` – new feature
 - `fix:` – bug fix
 - `chore:` – maintenance, refactor, docs, or test changes

Note: At the time of writing, the ArgoCD app target revision is `testing`.

1.4.2. Step-by-step

1. Clone the repository (first time only)

```
git clone https://github.com/sonaemc-kubernetes/lac1041-instore-orchestrator.git
cd lac1041-instore-orchestrator
```

2. Create a new branch from the ArgoCD target revision

```
git checkout <target-revision>
git pull origin <target-revision>
git checkout -b <your-branch-name>
```

Example (using `testing` as the current target revision):

```
git checkout testing
git pull origin testing
git checkout -b feature/update-esl-datapipeline-schedule
```

3. Edit the configuration file

Open `clusters/cluster-preproduction/values-pp.yaml` and apply the desired changes (schedule, filters, etc.).

4. Commit and push

```
git add clusters/cluster-preproduction/values-pp.yaml
git commit -m "chore: update esl datapipeline schedule and store filters"
git push origin <your-branch-name>
```

5. Open a Pull Request targeting the ArgoCD target revision

On GitHub, open a pull request from your branch targeting the ArgoCD target revision branch.

Once the PR is reviewed and merged, ArgoCD will detect the change and automatically sync the updated CronJob configuration to the cluster.